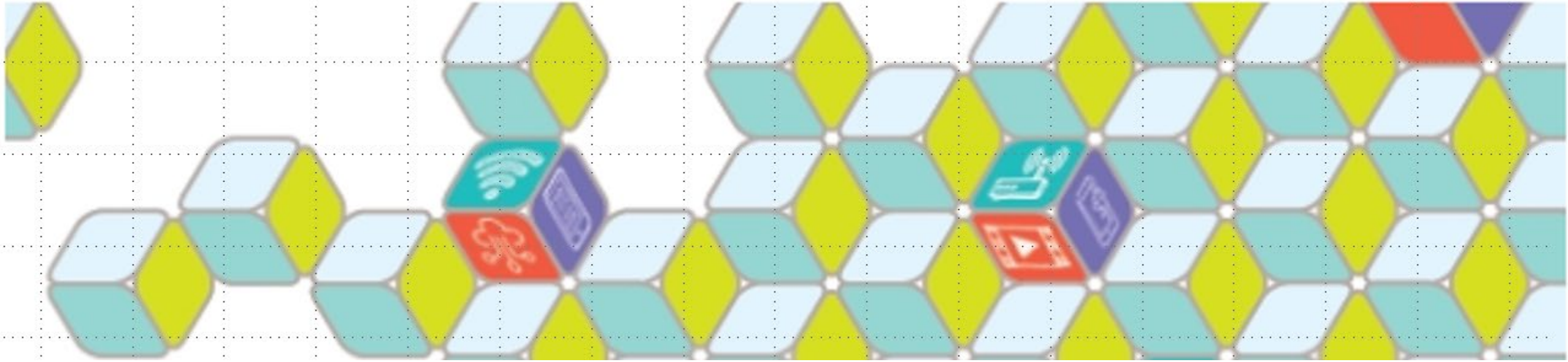


C Programming

(Programming Method using C, for Games)

Prof. Tae Soo Yun

Dept. of Game, International College, Dongseo University

DSU Dongseo University 동서대학교

Ch01_Tutotial Introduction

About C

- Developed by Dennis Ritchie of AT&T in the early 1970s
- B language -> C language
- Created out of necessity for developing the UNIX operating system
- From the beginning, it was designed as a language for professionals



The moment when Ken Thomson and Dennis Ritchie receive the National Medal of Technology from President Clinton.

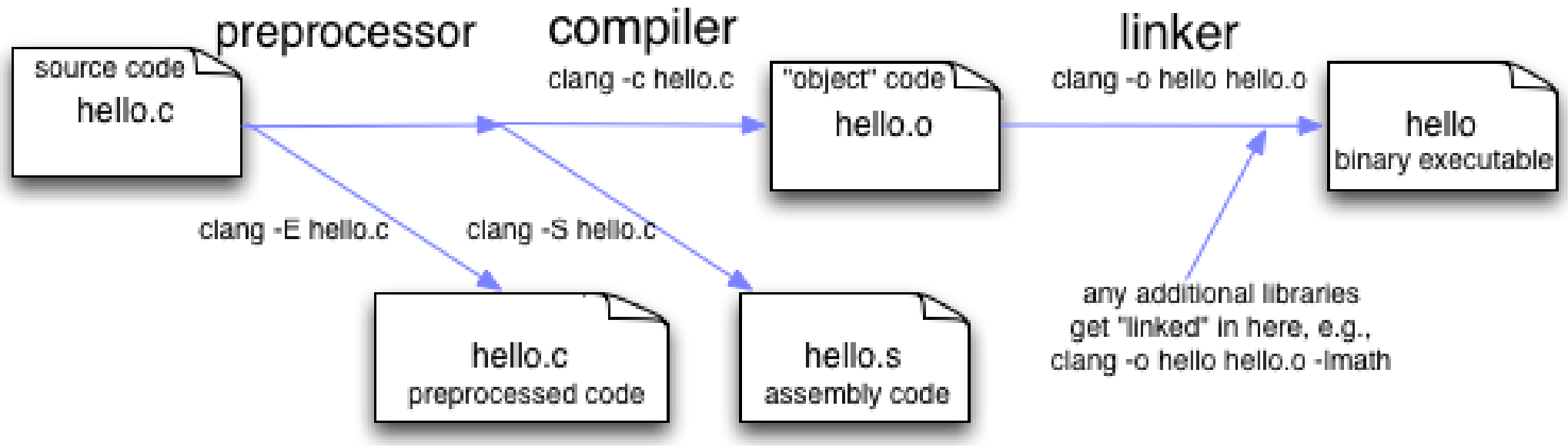


Characteristics of C Language

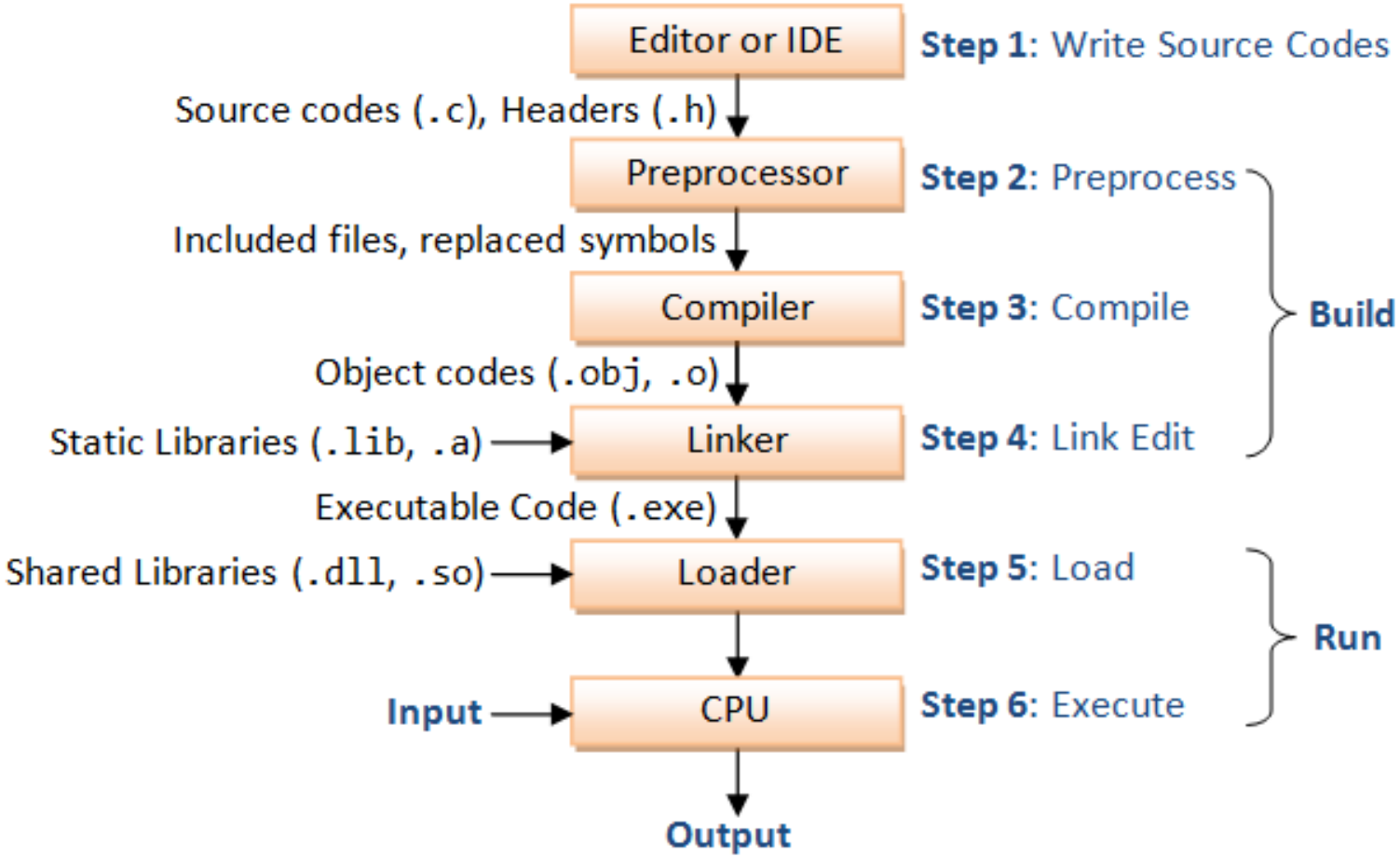
- Concise
- Efficient
- C language allows both low-level programming that directly controls hardware and high-level programming
- The C language is highly portable
- It is difficult for beginners to learn.

The process of C compilation

- clang is c-compiler running based on LINUX.
- we need to know the difference between compiler and interpreter



The process of C compilation

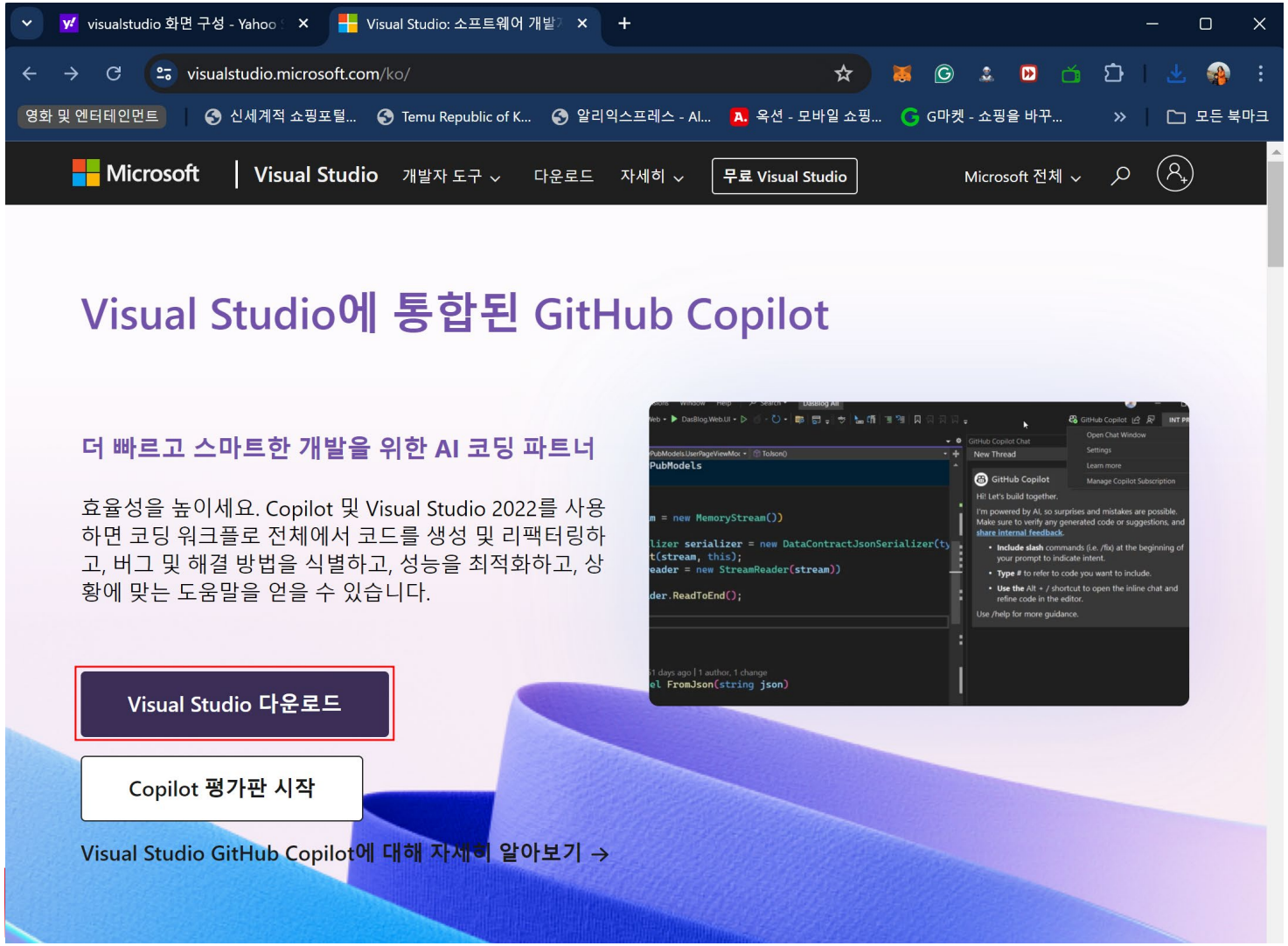


Install IDE(VS Studio 2022)

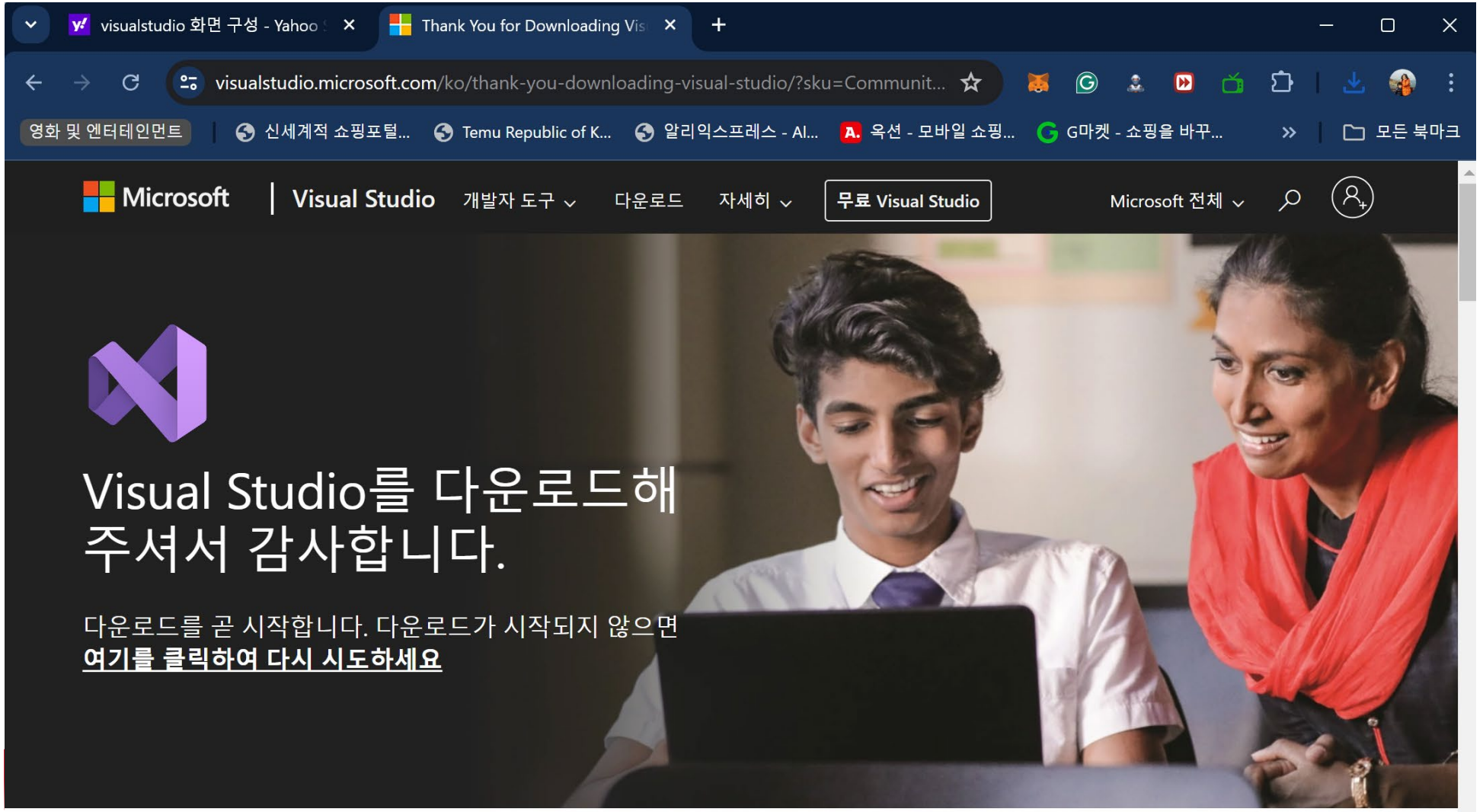
- IDE (Integrated Development Environment) is a software tool that integrate essential editor, compiler, debugger for developing program.
- Types : Eclipse (for Java), IntelliJ IDEA(for Java, Kotlin), Visual Studio (C, C#, ASP, crossplatform)
- Visual Studio xxx is provided by MS.
- Community version is free for students and beginner for development.

Install Visual Studio

- https://visualstudio.Microsoft.com/ko

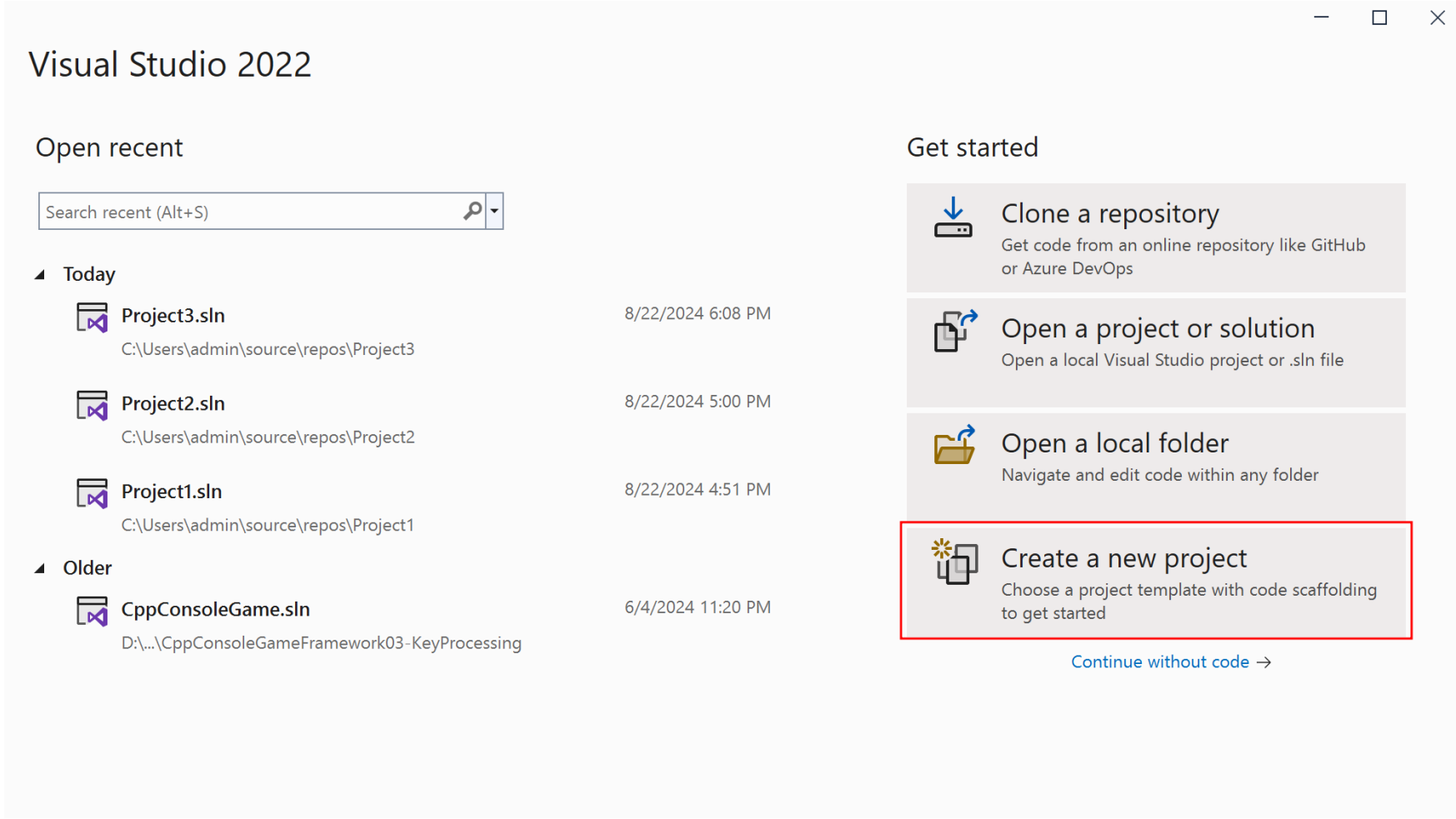


Installed VS 20xx on IC401

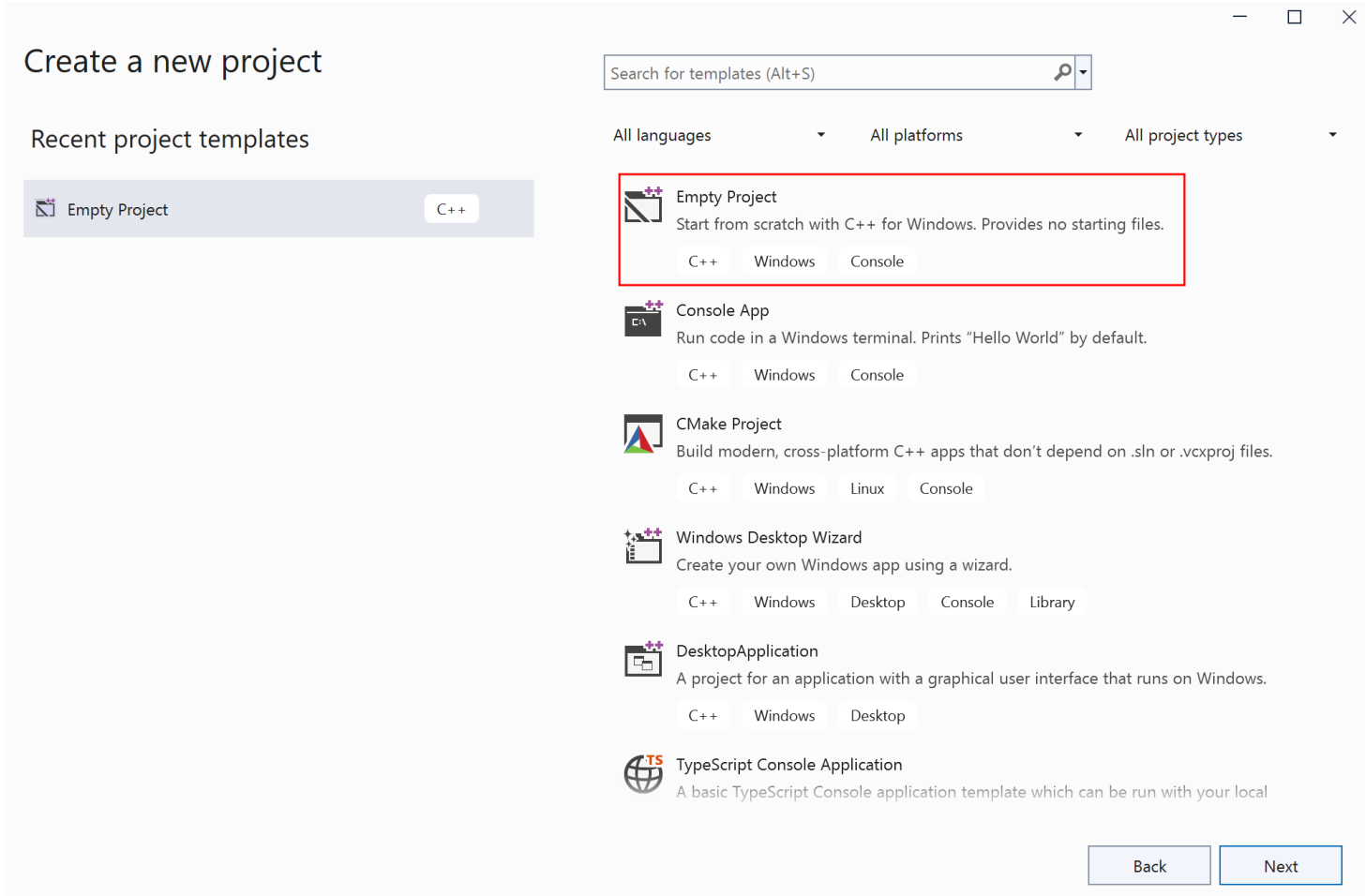

visualstudio 화면 구성 - Yahoo
Thank You for Downloading Vis
visualstudio.microsoft.com/ko/thank-you-downloading-visual-studio/?sku=Communit...
영화 및 엔터테인먼트
신세계적 쇼핑포털...
Temu Republic of K...
알리익스프레스 - Al...
옥션 - 모바일 쇼핑...
G마켓 - 쇼핑을 바꾸...
모든 북마크
Microsoft
Visual Studio
개발자 도구
다운로드
자세히
무료 Visual Studio
Microsoft 전체
Visual Studio를 다운로드해 주셔서 감사합니다.
다운로드를 곧 시작합니다. 다운로드가 시작되지 않으면
여기를 클릭하여 다시 시도하세요

1.1 Getting Started

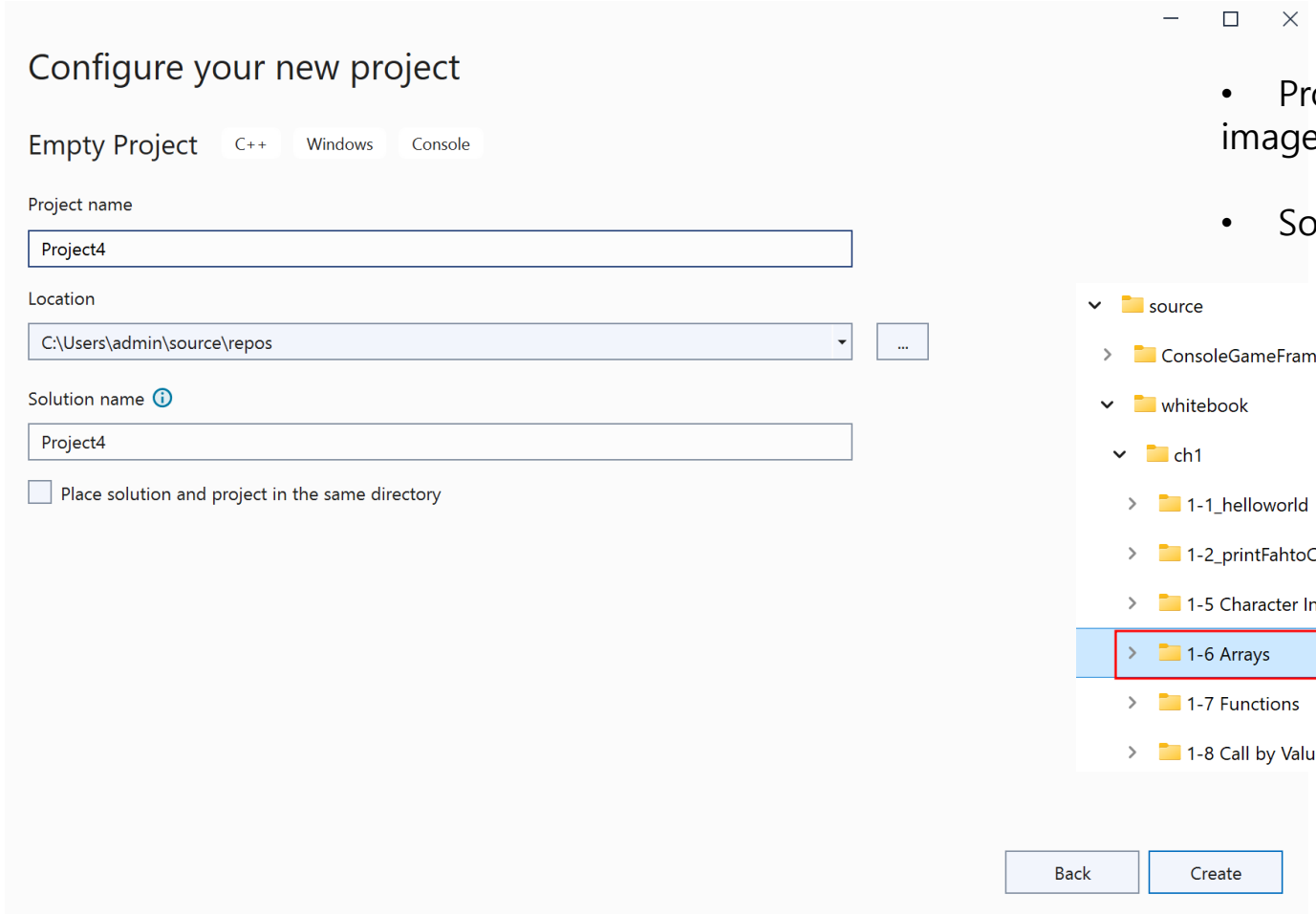
❖ start VS 2022, select Create a new project



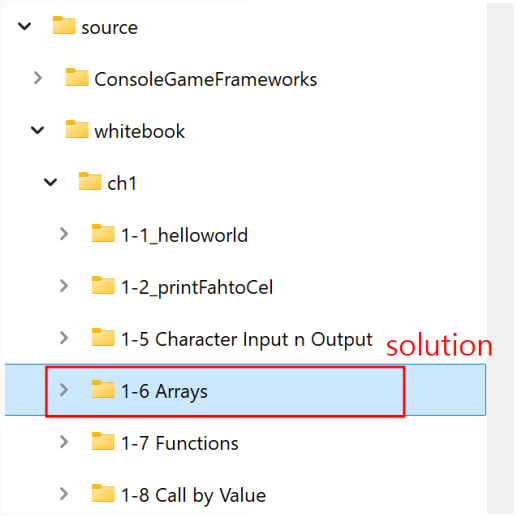
❖ select Empty project to do without a framework

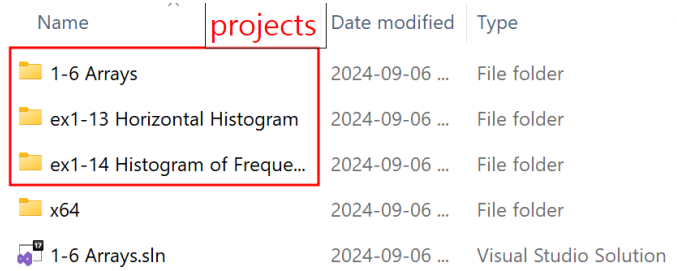


❖name your project, modify the location if you need to be changed..

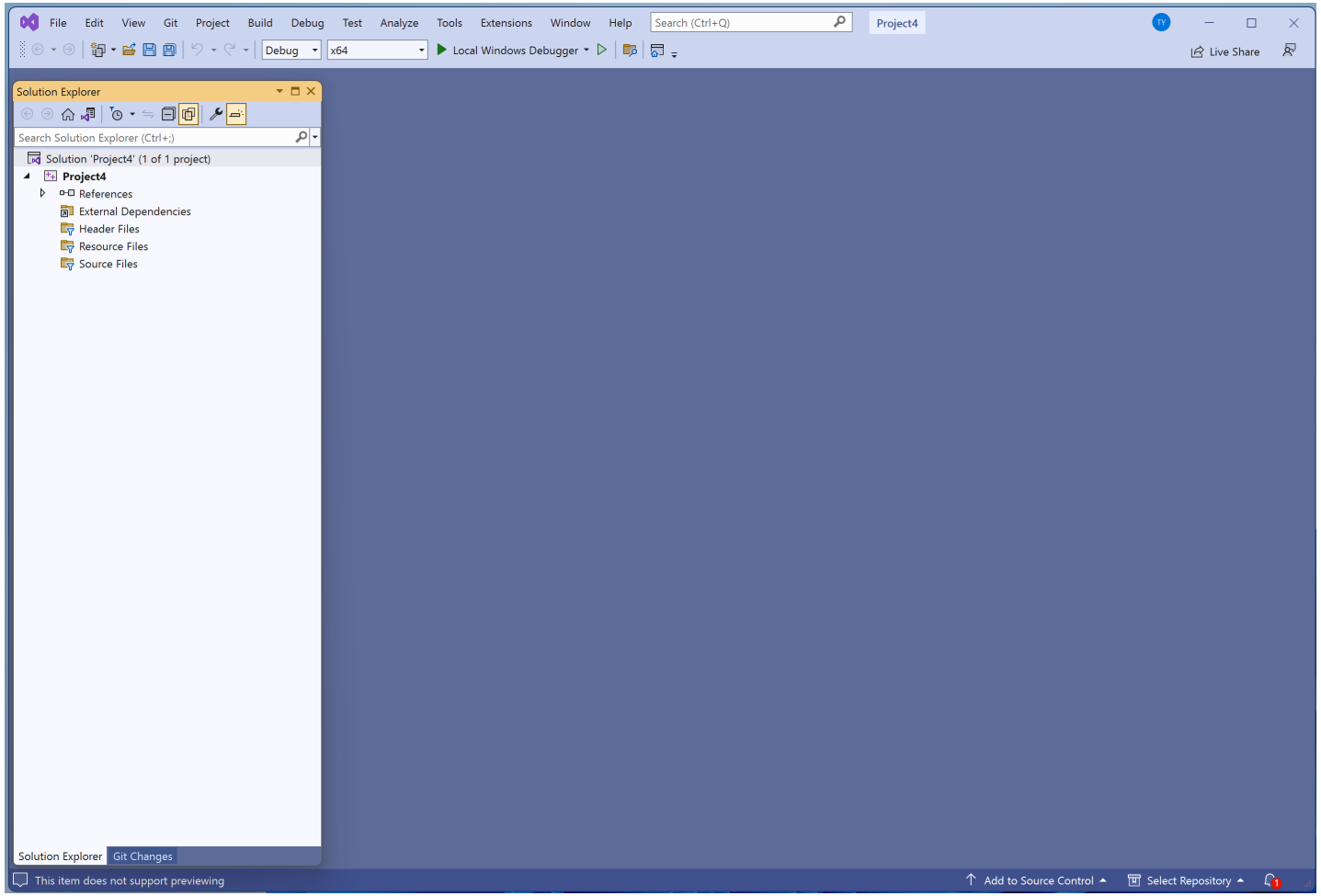


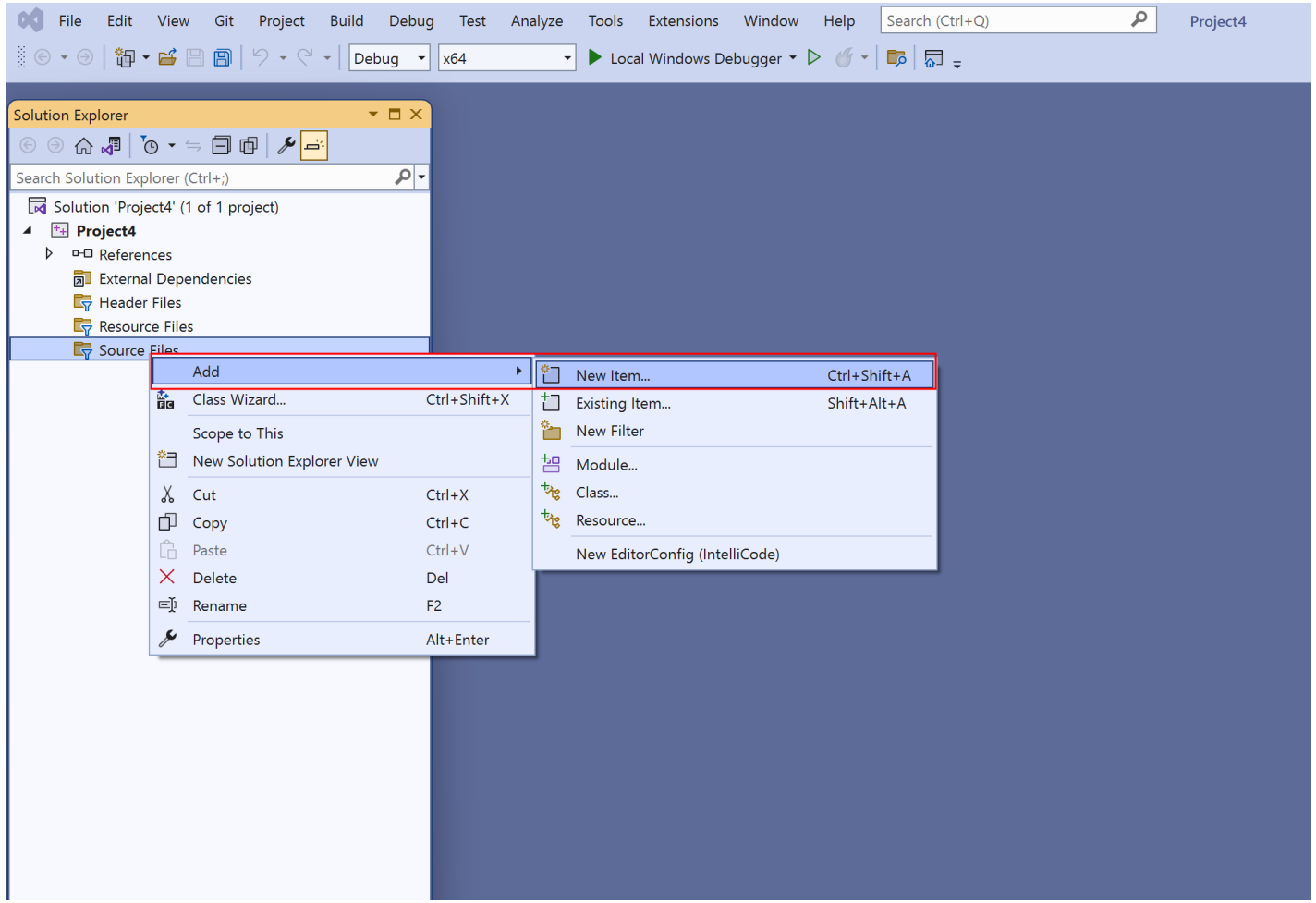
- Project : container that hold source code, icons, images, and data required to create executable files
- Solution: container includes several projects





❖ Solution explorer

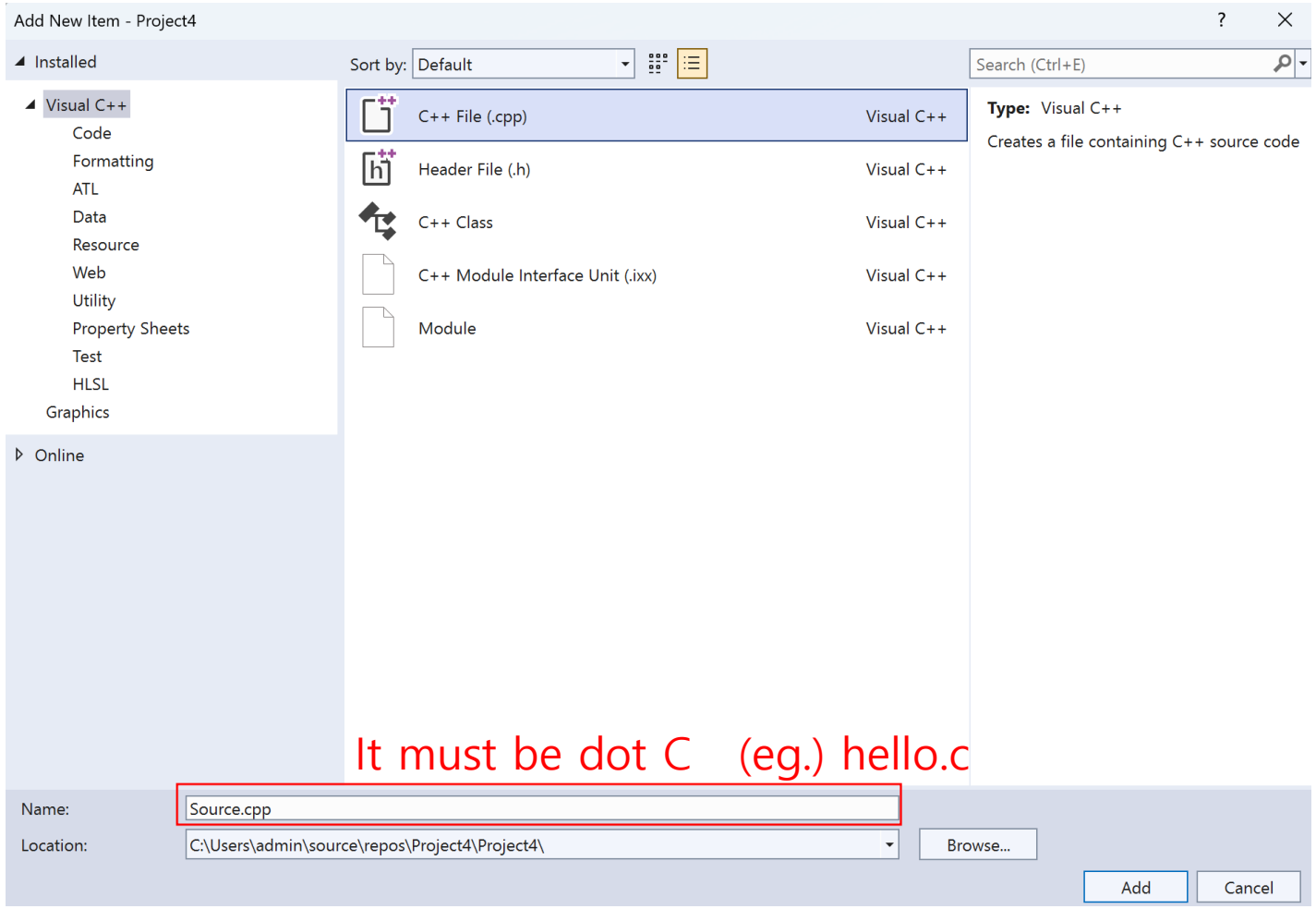




- In solution explorer window

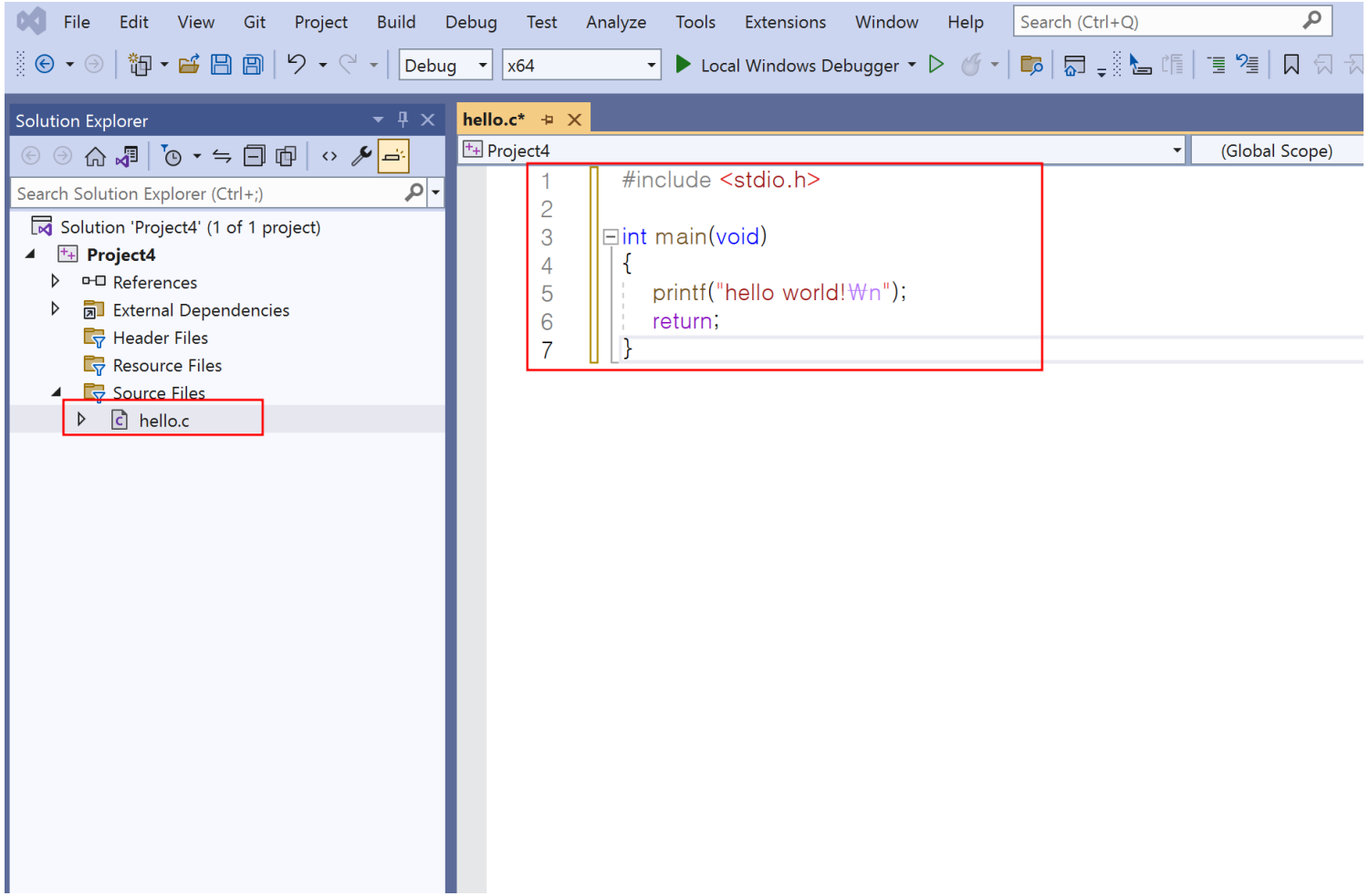
Right click on source files , Add, New item

❖Add hello.c file to the project



- From visual C++,
You can select any file it doesn't matter
- What really matters the most is the extension
It must be dot C (eg.) hello.c

❖Project with hello.c

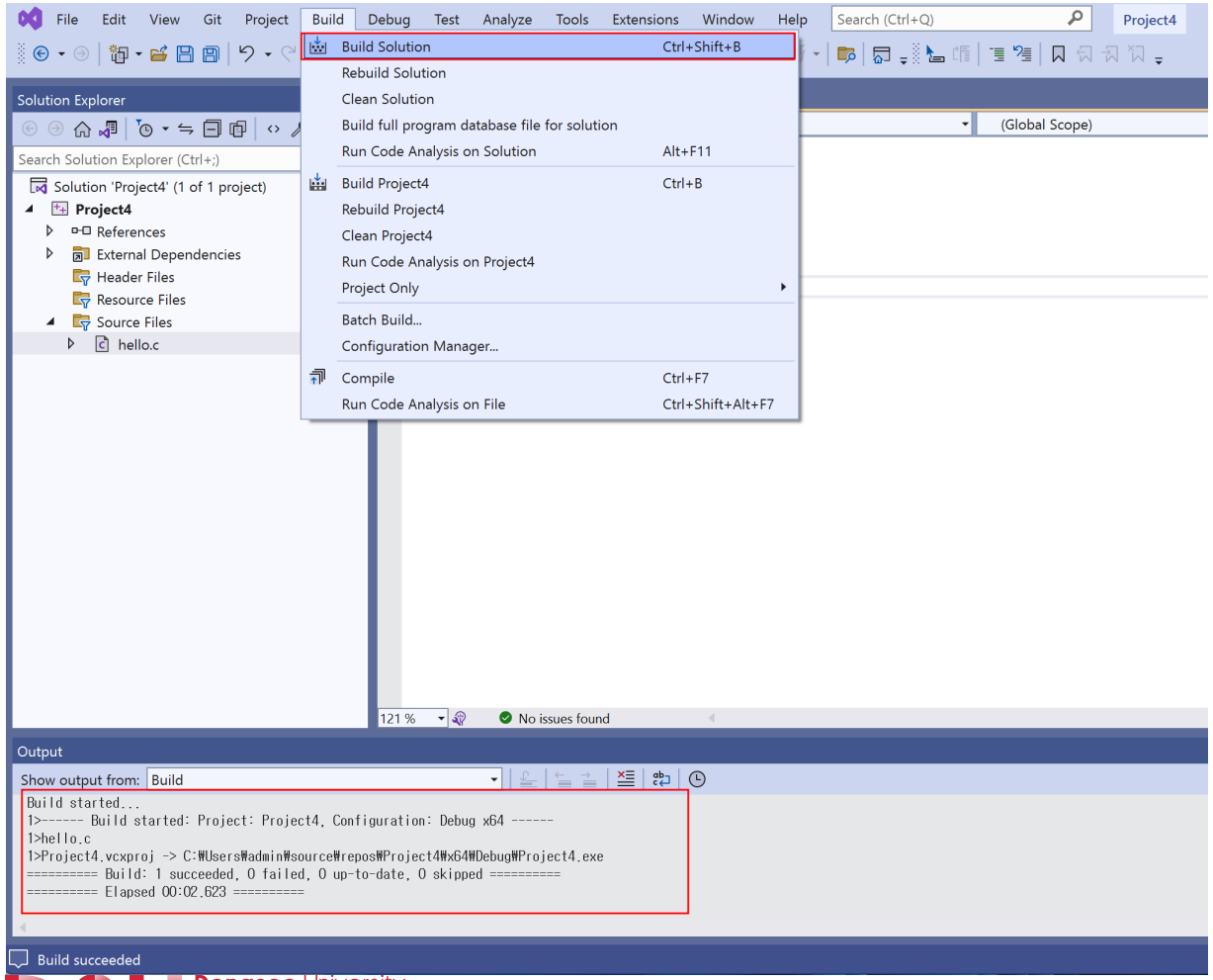


- The first program to write is the same for all languages:

```
Print the words
      hello, world
```

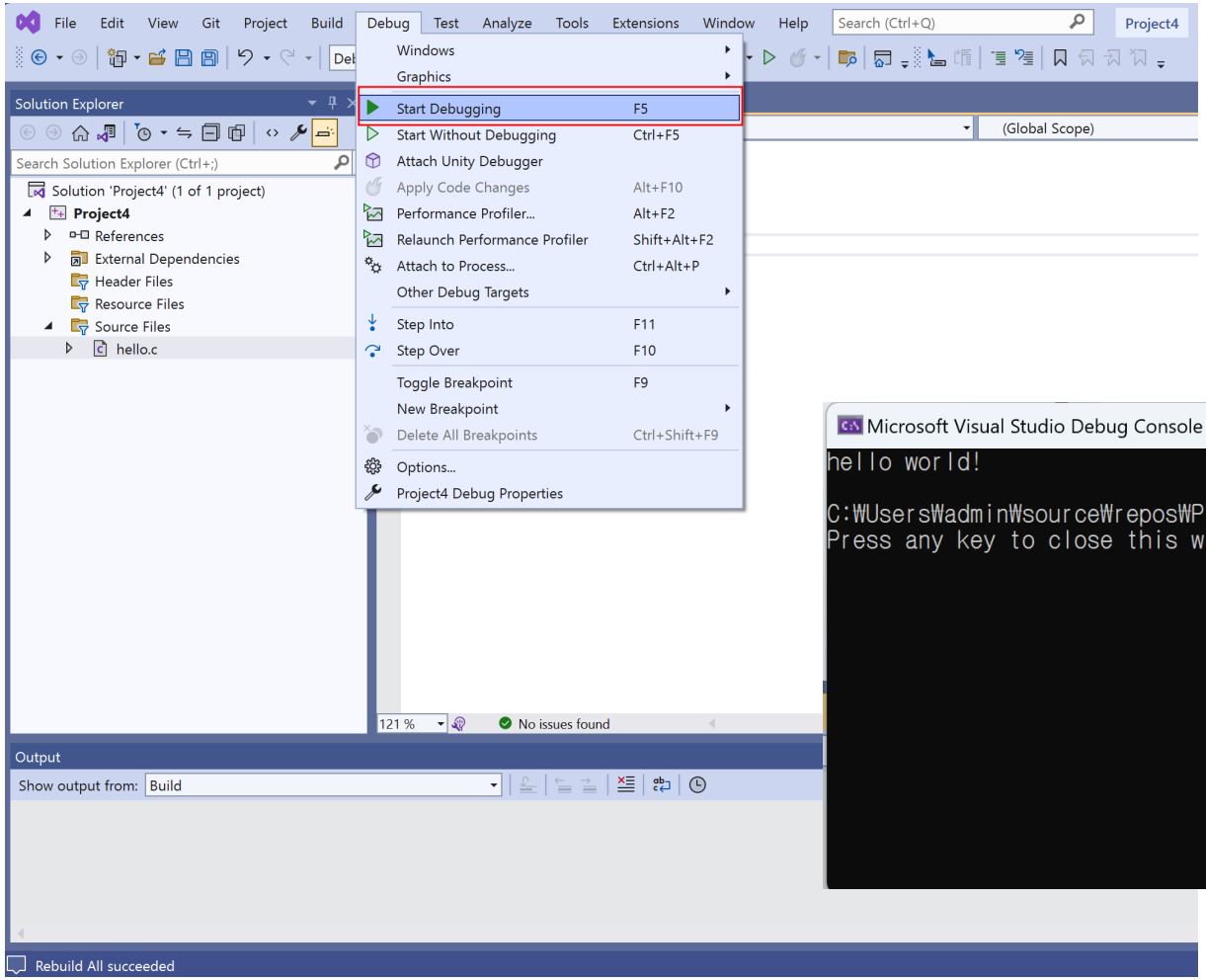
- write the code in editing window

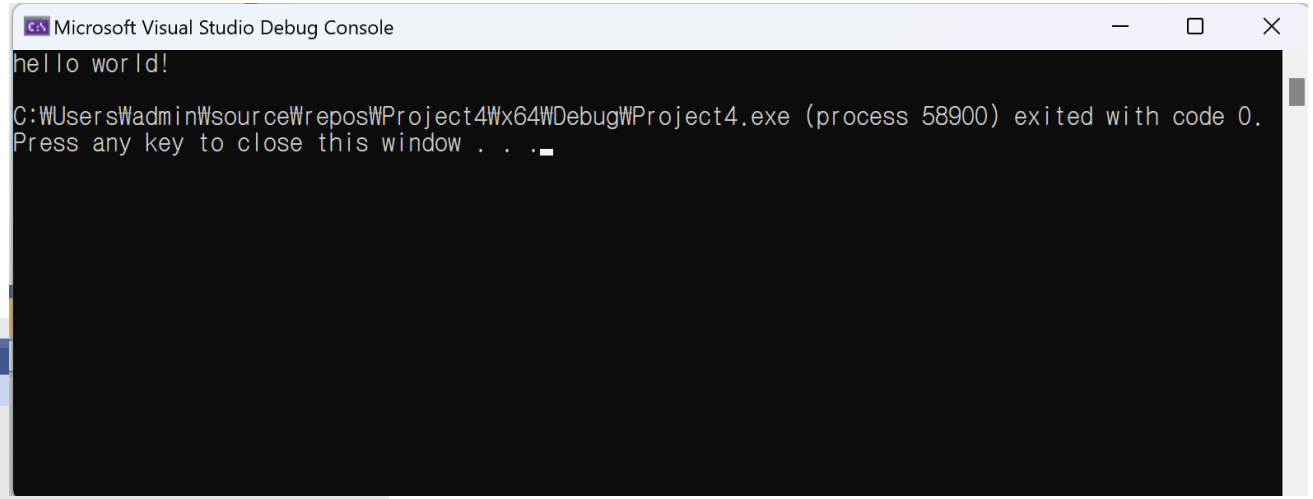
❖ Build (compile, link)



- In Build menu, select Build solution (or, Ctrl+Shift+B) to compile and link obj. and resources , produce hello.exe
- Output window display the result of compilation

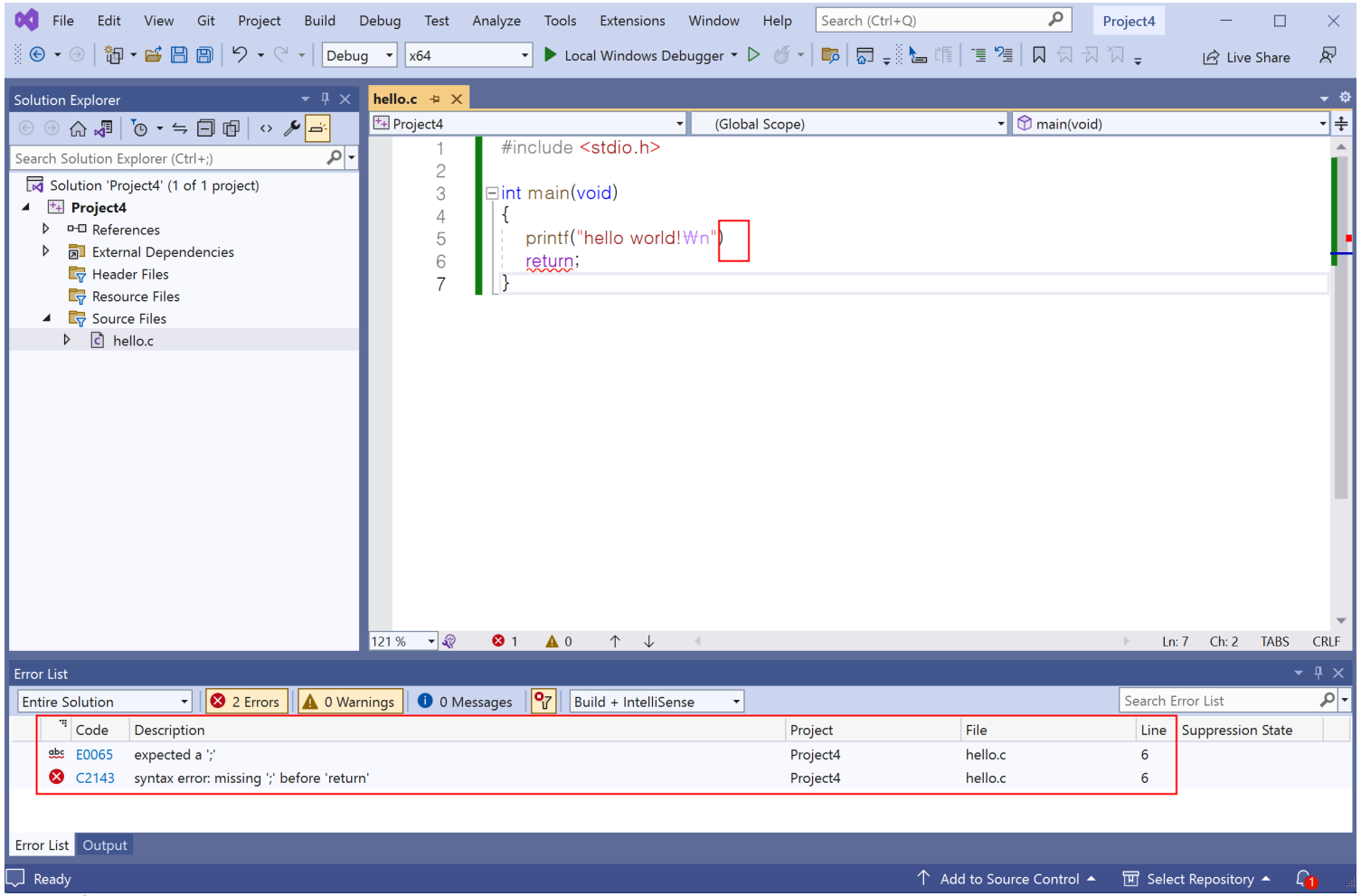
❖ Run program





- In Debug menu, select Start Without Debugging (or Ctrl F5), or Start Debugging (or F5)

❖Degugging Test



- remove a character or symbol.
- and run ctrl+F5 (F5)

1.1 Getting stated with hello.c

```
#include <stdio.h>

main()

{
    printf("hello, world\n");
}
```

include information about standard library

define a function named `main` *that receives no argument values*

statements of `main` *are enclosed in braces*

`main` *calls library function* `printf` *to print this sequence of characters;* `\n` *represents the newline character*

The first C program.

1.1 Getting started with hello.c

- A C program consists of functions and variables.
- Function contains statements & variables.
  - Statements specify the computing operation to be done
  - Variables store values during the computation.
- every C program must have a main() function
  - executed first at the beginning of program
  - main(), void main(void), int main(), …,
- #include <stdio.h> : preprocessor directive
  - Include information standard input/output library
  - Described in Appendix B : Standard Library
- Communicating data between functions
  - function_name(arg1, arg2, …., argn)
  - The parentheses after the function name surround the argument list.
  - printf() is a kind of function (function library), it calls the function printf with the argument "hello, world₩n"
- { } : punctuation, block notation
  - The statement of a function are enclosed in braces { }

```c
#include <stdio.h>

int main(void)
{
    printf("hello world!₩n");
    return;
}
```

The functions, types and macros of the standard library are declared in standard *headers*:

<assert.h>	<float.h>	<math.h>	<stdarg.h>	<stdlib.h>
<ctype.h>	<limits.h>	<setjmp.h>	<stddef.h>	<string.h>
<errno.h>	<locale.h>	<signal.h>	<stdio.h>	<time.h>

A header can be accessed by

```
#include <header>
```

1.1 Getting stated with hello.c

❖ escape sequence ₩, represents only a single character.

- Is a extensible mechanism for representing hard-to-type or invisible characters such as
  - ✓ ₩n newline, ₩t tab, ₩b for backspace, ₩" for the double quote, ₩₩ for the backslash itself
  - ✓ Explained in more detail in section 2.3

```
printf("hello, world
");
```

```
#include <stdio.h>

main()
{
    printf("hello, ");
    printf("world");
    printf("\n");
}
```

```
#include <stdio.h>

main()
{
    printf("hello, world\n");
}
```

exercise

- ex 1-1. Run the “hello, world” program on your system. Experiment with leaving out parts of the program, to see what error messages you get.

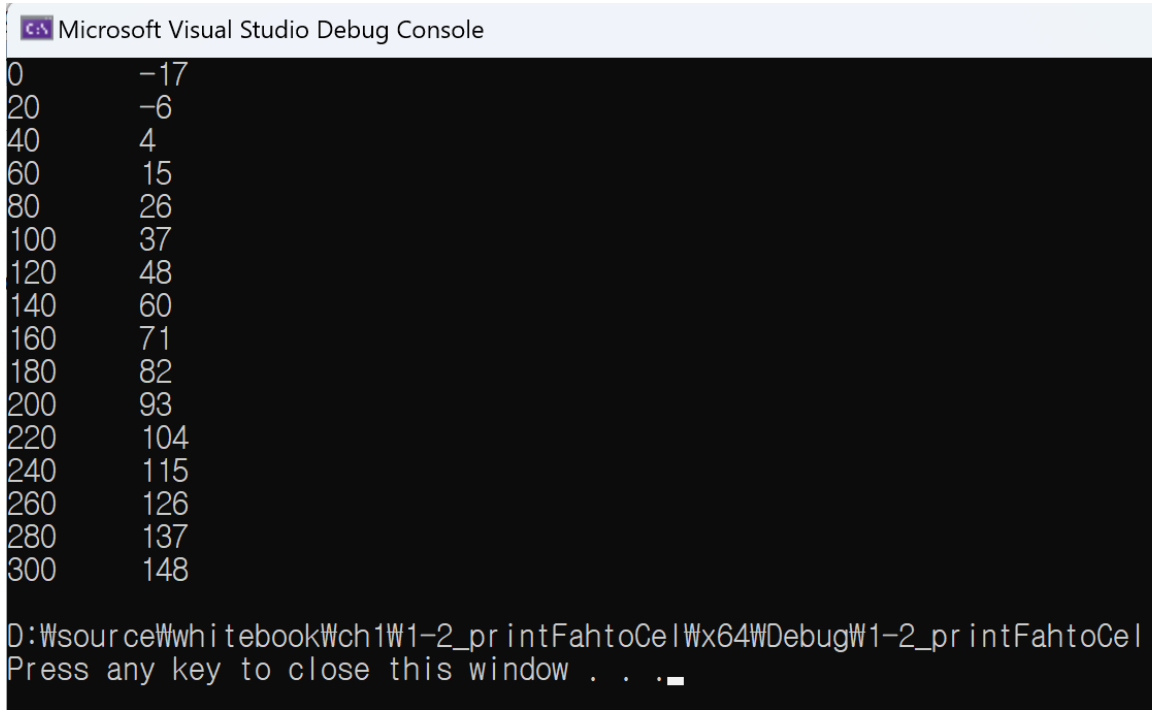
- ex 1-2. Experiment to find out what happens when printf’s argument string contains ₩c, where c is some character not listed above.

1.2 Variable and Arithmetic Expression

❖ program that print the Fahrenheit-celsius table using C= (5/9)(F-32)

- Comments, declarations, variables, arithmetic expression, loops, and formatted output.

```
1     #include <stdio.h>
2
3     /*print Fahrenheit-Celsisu table
4       for fahr = 0, 20, ..., 300 */
5
6     main()
7     {
8       int fahr, celsius;
9       int lower, upper, step;
10
11      lower = 0;          // lower limit of temperature table
12      upper = 300;        // upper limit
13      step = 20;          //  step size
14
15      fahr = lower;
16      while (fahr <= upper) {
17        celsius = 5 * (fahr - 32) / 9;
18        printf("%d₩t%d₩n", fahr, celsius);
19          fahr = fahr + step;
20      }
21    }
22
```



1.2 Variable and Arithmetic Expression

- comment : r.h.s are comment, which explains what the program does.
  - /* … */ is a block comment, // line comment
- variable is a storage or memory that store a value to be assigned or calculated during program executing
  - All var. must be declared before they are used.
  - Consists of a type name and a list of variables, such as

    Int fahr, Celsius;
- several data type in C
  - **Character**: char(1), **integer** : short(2), Int(4), long(4),long long(8)
    floating point : float(4), double(8), long double(8)
  - The size of these obj. are machine dependent. To be learned next chapter.
- Assignment statement
  - variable = value | expression | var ;
  - ex) lower = 0;

```
/*print Fahrenheit-Celsisu table
   for fahr = 0, 20, ..., 300 */

main()
{
   int fahr, celsius;
   int lower, upper, step;

   lower = 0;          // lower limit of temperature table
   upper = 300;        // upper limit
   step = 20;          //  step size
```

```
printf("char ₩t₩t-> %d ₩n", sizeof(char));
printf("short ₩t₩t-> %d ₩n", sizeof(short));
printf("int ₩t₩t-> %d ₩n", sizeof(int));
printf("long ₩t₩t-> %d ₩n", sizeof(long));
printf("long long ₩t-> %d ₩n", sizeof(long long));
printf("float ₩t₩t-> %d ₩n", sizeof(float));
printf("double ₩t₩t-> %d ₩n", sizeof(double));
printf("long double ₩t-> %d ₩n", sizeof(long double));
```

1.2 Variable and Arithmetic Expression

❖ while loop used to represent repeated operation.

```
while (fahr <= upper) {
    celsius = 5 * (fahr - 32) / 9;
    printf("%d\t%d\n", fahr, celsius);
    fahr = fahr + step;
}
```

❖ printf(); described in detail in ch7.

- printf("argument specifier1, specifier2 ..., specifier n", argument1, 2, ..., n);
- printf("%d₩t%d₩n", fahr, celsius);
- printf("%3d %6d₩n", fahr, celsius); // for right-justified
- printf("%3d %6.1f₩n", fahr, celsius); // for floating-point

```
0   -17        0   -17        0  -17.8
20   -6       20    -6       20   -6.7
40    4       40     4       40    4.4
60   15       60    15      ...
80   26       80    26
100  37      100    37
              ...
```

❖ various argument specifier (placeholder)

- %o for octal, %x for hexadecimal,

  %c for character, %s for string, %u for unsigned
- %ld for long, %lld for long long
- %f for float, %lf for double

%d	print as decimal integer
%6d	print as decimal integer, at least 6 characters wide
%f	print as floating point
%6f	print as floating point, at least 6 characters wide
%.2f	print as floating point, 2 characters after decimal point
%6.2f	print as floating point, at least 6 wide and 2 after decimal point

exercise

❖ ex 1-3. Modify the temperature conversion program to print a heading above the table.

❖ ex 1-4. Write a program to print the corresponding Celsius to Fahrenheit table.

1.3 the for statement

❖ compare the two statements

- Produces the same result.
- Whether to use while or for is up to your preference.
- for loop format

```
for ( initialization ; termination condition; increment step )
    Statements;
```

- while loop format

```
initialization; flag;
while ( termination cond | flag) {
    statements;
    increment step;
}
```

```
for (fahr = 0; fahr <= 300; fahr = fahr + 20) {
    celsius = 5 * (fahr - 32) / 9;
    printf("%3d  %6.1f\n", fahr, celsius);
}
```

```
fahr = lower;
while (fahr <= upper) {
    celsius = 5 * (fahr - 32) / 9;
    printf("%d\t%d\n", fahr, celsius);
    fahr = fahr + step;
}
```

❖ tips for while loop : use flag

```
fahr = lower;
while (fahr <= upper) {
   celsius = 5 * (fahr - 32) / 9;
   printf("%d\t%d\n", fahr, celsius);
   fahr = fahr + step;
}
```

```
fahr = lower;
flag = 1;
while (flag) {
   celsius = 5 * (fahr - 32) / 9;
   printf("%d\t%d\n", fahr, celsius);
   fahr = fahr + step;
   if(fahr <= upper) flag = 0;
}
```

Exercise

❖ ex 1-5. Modify the temperature conversion program to print the table in reverse order, that is , from 300 degree to 0.

1.4 Symbolic Constants

❖ so that easy to modify or maintain the program later. symbolic name or symbolic constant is defined as

```
#define name replacement_text
```

❖ any occurrence of *name* will be replaced by the corresponding *replacement_text*

- Replacement _text can be any sequence of char., not to limited to numbers.

```
#include <stdio.h>

#define   LOWER  0       /* lower limit of table */
#define   UPPER  300     /* upper limit */
#define   STEP   20      /* step size */

/* print Fahrenheit-Celsius table */
main()
{
    int fahr;

    for (fahr = LOWER; fahr <= UPPER; fahr = fahr + STEP)
        printf("%3d %6.1f\n", fahr, (5.0/9.0)*(fahr-32));
}
```

1.5 Character Input and Output

- learn about methods for processing character data.
- The standard library provides for reading or writing one character at a time, getchar() and putchar()

```
c = getchar();    /* reads the next input character from a text
                     stream returns that as its value*/
putchar(c);       /* prints the contents of the integer var. c
                     as a character */
```

1.5 Character Input and Output

❖ 1.5.1 File Copying : A program that copies its input to its output one character at a time

- Where is the end of this file(EOF) ?
- How to input EOF into text stream?
- EOF is an integer defined in <stdio.h>

```
Read a character
While( character is not end-of-file indicator)
   Output the character just read
   Read a character
```

```
#include <stdio.h>

// copy input to output ; 2nd version
main()
{
    int c;

    while((c = getchar()) != EOF) {
        putchar(c);
    }
}
```

```
#include <stdio.h>

// copy input to output ; 1st version
main()
{
    int c;

    c= getchar();
    while(c != EOF) {
        putchar(c);
        c = getchar();
    }
}
```

1.5 Character Input and Output

❖ 1.5.2 Character Counting

- It is similar to the copy program.
- Look at the ++nc, needs identify prefix operator ++nc, and postfix operator nc++;

(if n =5, x = n++; vs x = ++n; what is the val. of x ? respectively)

```
#include <stdio.h>

// count characters in input ; 1st version
main()
{
    long nc;

    nc = 0;
    while(getchar() != EOF)
        ++nc ;
    printf("%ld\n", nc);
}
```

```
#include <stdio.h>

// count characters in input ; 2nd version
main()
{
    double nc;

    for(nc =0; getchar()!= EOF; ++nc)
        ;           // null statement
    printf("%.0f\n", nc);
}
```

1.5 Character Input and Output

❖ 1.5.3 Line Counting

- A character written between single quotes represents an integer value equal to the numerical values of the character.
- 'A' is a character constant ; 65 , refer to ASCII table
- '₩n' stands for the value of the newline character, 10

```
#include <stdio.h>

// count lines in input
main()
{
    int c, nl;

    nl = 0;
    while((c=getchar()) != EOF)
        if ( c == '\n') ++nl;
    printf("%d\n", nl);
}
```

Exercise 1-8. Write a program to count blanks, tabs, and newlines. □

Exercise 1-9. Write a program to copy its input to its output, replacing each string of one or more blanks by a single blank. □

Exercise 1-10. Write a program to copy its input to its output, replacing each tab by `\t`, each backspace by `\b`, and each backslash by `\\`. This makes tabs and backspaces visible in an unambiguous way. □

❖ 1.5.4 Word Counting

- How can we write a program count words from input character stream?
- Utilize state variable
- Operator || means OR, && AND

```c
#include <stdio.h>

#define IN 1   // inside a word
#define OUT 0  // outside a word

// count lines, words, and characters in input
main()
{
    int c, nl, nw, nc, state;

    state = OUT;
    nl = nw = nc = 0;
    while ((c = getchar()) != EOF) {
        ++nc;
        if (c == '₩n') ++nl;
        if (c == ' ' || c == '₩n' || c == '₩t') state = OUT;
        else if (state == OUT) {
            state = IN;
            ++nw;
        }
    }
    printf("%d %d %d₩n", nl, nw, nc);
}
```

Exercise 1-11. How would you test the word count program? What kinds of input are most likely to uncover bugs if there are any? □

Exercise 1-12. Write a program that prints its input one word per line. □

❖ 1.6 Arrays

- An array is a collection of multiple occurrences of the same data

```
#include <stdio.h>

/* count digits, white space, others */
void main() {
    int c, i, nwhite, nother;
    int ndigit[10];

    nwhite = nother = 0;
    for (i = 0; i < 10; ++i)
        ndigit[i] = 0;

    while ((c = getchar()) != EOF) {
        if (c >= '0' && c <= '9')
            ++ndigit[c - '0'];
        else if (c == ' ' || c == '₩n' || c == '₩t')
            ++nwhite;
        else
            ++nother;
    }
    printf("digits =");
    for (i = 0; i < 10; ++i)
        printf(" %d", ndigit[i]);
    printf(", white space = %d, other = %d₩n", nwhite, nother);

}
```

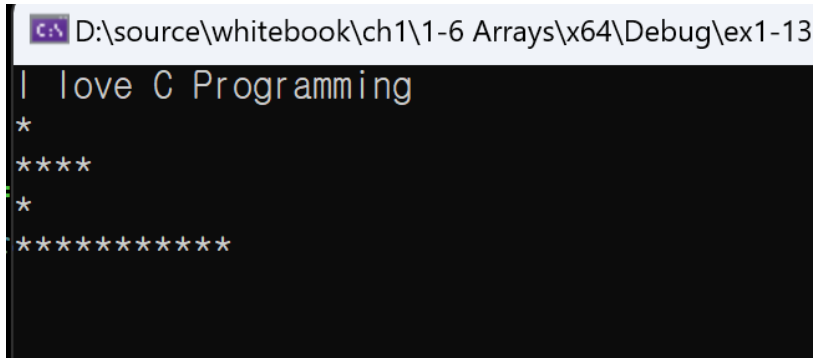
Microsoft Visual Studio Debug Console

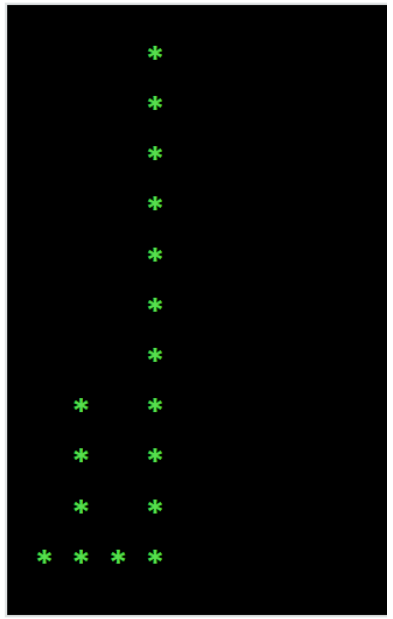
```
3 3 3 3 3 3 3 3 3 3
^Z
digits = 0 0 0 9 0 0 0 0 0 0, white space = 10, other = 0

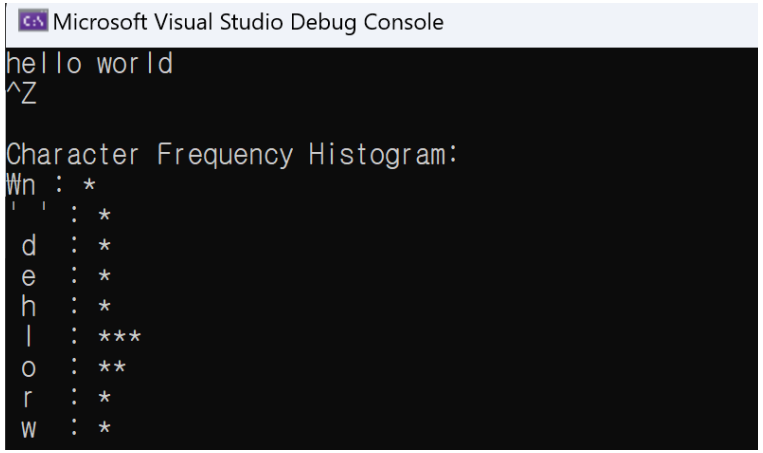
D:₩source₩whitebook₩ch1₩sec1-5₩x64₩Debug₩sec1-6 Arrays.exe (process 44388) exi
To automatically close the console when debugging stops, enable Tools->Options
le when debugging stops.
Press any key to close this window . . .
```

❖ **Exercise 1-13.** Write a program to print a histogram of the lengths of words in its input. It is easy to draw the histogram with the bars horizontal; a vertical orientation is more challenging. □

Exercise 1-14. Write a program to print a histogram of the frequencies of different characters in its input. □







1.7 Functions

- a function provides a convenient way to encapsulate some computation, which can then be used without worrying about its implementation.
- Write a function power(m,n) to raise an integer m to a positive integer power n.
- a function definition

```
Return-type function-name(parameter declarations
, if any)
{
    declarations
    statements
}
```

```c
#include <stdio.h>

int power(int base, int n);

/* test power function */
int main() {
    int i;
    for (i = 0; i < 10; ++i)
        printf("%d %d %d₩n", i, power(2, i), power(-3, i));
    return 0;
}

/* power: raise base to n-th power; n >= 0 */
int power(int base, int n) {
    int i, p;
    p = 1;
    for (i = 1; i <= n; ++i)
        p = p * base;
    return p;
}
```

❖ exercise 1-15. Rewrite the temperature conversion program of section 1.2 to use a function for conversion.

1.8 Arguments – call by value

❖ parameter passing method

- Call by value
- Call by reference (address)

```c
#include <stdio.h>

int add(int, int );

int main()
{
    int a, b, sum;

    a = b = 10;
    sum = add(a, b);

    printf("the sum of a, b = % d₩n", sum);
    printf("a = %d, b = %d₩n", a, b);
    return 0;

}

int add(int aa, int bb) {
    aa += 1; bb += 1;
    return(aa + bb);
}
```

Microsoft Visual Studio Debug Console

```
the sum of a, b =  22
a = 10, b = 10
```

```c
#include <stdio.h>

int add(int *, int *);

int main()
{
    int a, b, sum;

    a = b = 10;
    sum = add(&a, &b);

    printf("the sum of a, b = % d₩n", sum);
    printf("a = %d, b = %d₩n", a, b);
    return 0;

}

int add(int *aa, int *bb) {
    *aa += 1; *bb += 1;
    return(*aa + *bb);
}
```

Microsoft Visual Studio Debug Console

```
the sum of a, b =  22
a = 11, b = 11
```

1.9 Character Arrays

❖ write a program that reads a set of text lines and prints the longest.

❖ algorithm pseudo code.

```
While (there's another line)
    if (it's longer than the previous longest)
        save it
        save its length
Print longest line
```

❖ Take a look at parameter passing method carefully

- Int mgetline(char line[ ], int maxline); called function
- mgetline(line, MAXLINE) caller

1.9 Character Arrays

```c
#include <stdio.h>

#define MAXLINE 1000    /* maximum input line length */

int mgetline(char line[], int maxline);
void copy(char to[], char from[]);

/* print the longest input line */
int main() {
    int len;                /* current line length */
    int max;                /* maximum length seen so far */
    char line[MAXLINE];     /* current input line */
    char longest[MAXLINE];  /* longest line saved here */
    max = 0;
    while ((len = mgetline(line, MAXLINE)) > 0)
        if (len > max) {
            max = len;
            copy(longest, line);
        }
    if (max > 0) /* there was a line */
        printf("%s", longest);
    return 0;
}
```

```c
/* mgetline: read a line into line, return length */
int mgetline(char line[], int maxline) {
    int c, i;
    for (i = 0; i < maxline - 1 && (c = getchar()) != EOF && c != '\n'; ++i)
        line[i] = c;
    if (c == '\n') {
        line[i] = c;
        ++i;
    }
    line[i] = '\0';
    return i;
}

/* copy: copy 'from' into 'to'; assume to is big enough */
void copy(char to[], char from[]) {
    int i;
    i = 0;
    while ((to[i] = from[i]) != '\0')
        ++i;
}
```

Exercise 1-16. Revise the main routine of the longest-line program so it will correctly print the length of arbitrarily long input lines, and as much as possible of the text. □

Exercise 1-17. Write a program to print all input lines that are longer than 80 characters. □

Exercise 1-18. Write a program to remove trailing blanks and tabs from each line of input, and to delete entirely blank lines. □

Exercise 1-19. Write a function `reverse(s)` that reverses the character string `s`. Use it to write a program that reverses its input a line at a time. □

1.10 External Variables and Scope

- ❖ local variables vs., global variables
- ❖ local variables
  - Local variables are defined within a function.
  - comes into existence only when the function is called,
  - and disappears when the function is exited. Known as automatic variable.
- ❖ global variables
  - Global variables are defined external to all functions,
  - can be accessed by name by any function
  - remain in existence permanently, they retain their values even after the functions return.
  - Have to declare in each function that wants to access it, but can be omitted if global variables defined within same source file.
- ❖ alternative to automatic variables
  - Possible to define variables that are ***external*** to all functions.
  -

```c
#include <stdio.h>

#define MAXLINE 1000   /* maximum input line size */
int max;               /* maximum length seen so far */
char line[MAXLINE];    /* current input line */
char longest[MAXLINE]; /* longest line saved here */
int mgetline(void);

void copy(void);

/* print longest input line; specialized version */
int main() {
    int len;
    extern int max;
    extern char longest[];
    max = 0;
    while ((len = mgetline()) > 0)
        if (len > max) {
            max = len;
            copy();
        }
    if (max > 0) /* there was a line */
        printf("%s", longest);
    return 0;
}
```

```c
/* getline: specialized version */
int mgetline(void) {
    int c, i;
    extern char line[];
    for (i = 0; i < MAXLINE - 1 && (c = getchar()) != EOF && c != '₩n'; ++i)
        line[i] = c;
    if (c == '₩n') {
        line[i] = c;
        ++i;
    }
    line[i] = '₩0';
    return i;
}

/* copy: specialized version */
void copy(void) {
    int i;
    extern char line[], longest[];
    i = 0;
    while ((longest[i] = line[i]) != '₩0')
        ++i;
}
```

Exercise 1-20. Write a program `detab` that replaces tabs in the input with the proper number of blanks to space to the next tab stop. Assume a fixed set of tab stops, say every *n* columns. Should *n* be a variable or a symbolic parameter? □

Exercise 1-21. Write a program `entab` that replaces strings of blanks by the minimum number of tabs and blanks to achieve the same spacing. Use the same tab stops as for `detab`. When either a tab or a single blank would suffice to reach a tab stop, which should be given preference? □

Exercise 1-22. Write a program to "fold" long input lines into two or more shorter lines after the last non-blank character that occurs before the *n*-th column of input. Make sure your program does something intelligent with very long lines, and if there are no blanks or tabs before the specified column. □

Exercise 1-23. Write a program to remove all comments from a C program. Don't forget to handle quoted strings and character constants properly. C comments do not nest. □

Exercise 1-24. Write a program to check a C program for rudimentary syntax errors like unbalanced parentheses, brackets and braces. Don't forget about quotes, both single and double, escape sequences, and comments. (This program is hard if you do it in full generality.) □